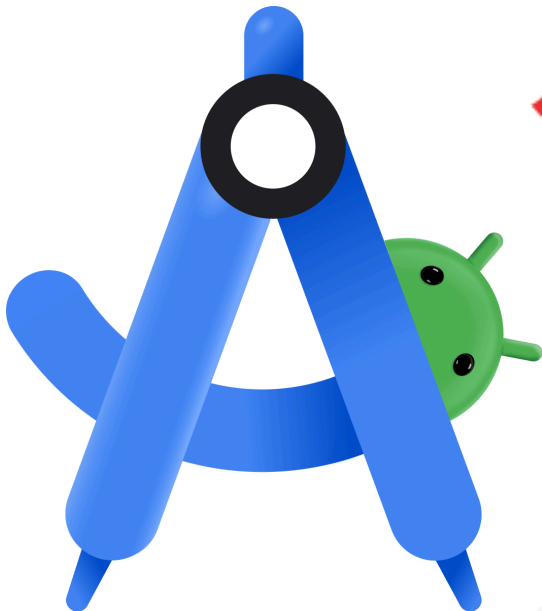


BOOKJOURNAL APP



Java™



Egilea: Ainhize Arrese Ugarte

AURKIBIDEA

1. SARRERA.....	3
1.1. Erabilitako teknologia.....	3
2. APLIKAZIOAREN EZAUGARRIAK.....	4
2.1 Autentifikazio Sistema.....	4
2.1.1 Hasierako Pantaila (FirstFragment).....	4
2.1.2 Erregistro Pantaila (SingUpFragment).....	5
2.1.3 Saio-hasiera Pantaila (LoginFragment).....	5
2.1.4 Saio-amaiera (Logout).....	6
2.2 Nabigazioa eta Menuak.....	6
2.2.1 Nabigazioaren Egitura.....	6
2.2.2 Menu Nagusiak.....	6
2.2.3 Nabigazioaren kudeaketa.....	7
2.3 Irakurketa Egunerokoa (Reading Log).....	8
2.3.1 Liburuen Bilaketa.....	8
2.3.2 Emaizen txartelak.....	8
2.3.3 Irakurketa datuen sarrera.....	9
2.3.4 Irakurketa erregistroen zerrenda.....	10
2.4 Irakurketa Sistema (Reading System).....	11
2.4.1 Atal honen helburua.....	11
2.4.2 Pantailaren diseinua.....	11
2.4.3 Funtzionamendua.....	12
2.4.4 Firestore-ko egitura.....	12
2.5 Desio-zerrenda (Wishlist).....	13
2.5.1 Liburuen bilaketa.....	13
2.5.2 Desio-zerrendaren kudeaketa.....	14
2.6 Liburuen Informazioa (Book Information).....	16
2.6.1 Bilaketa.....	16
2.6.2 Emaizen txartelak.....	17
2.6.3 Sinopsien Karga.....	17
2.6.4 Xehetasunen Pantaila.....	19
2.7 Liburu Gogokoenak (Favorite Books).....	20
2.7.1 Funtzionamendu Automatikoa.....	20
2.7.2 ReadingLog-etik FavoriteBooks-era.....	21
2.7.3 Pantailaren diseinua.....	22
3. APLIKAZIOKO KODIGOAREN EGITURA.....	23
4. OINARRIZKO FUNTZIONAMENDUA.....	24
4.1. Autentifikazio-fluxua.....	24
4.3. Nabigazioaren kudeaketa.....	24
4.3. Open Library API integrazioa.....	25
4.4. Firestore datu-egitura.....	25
5. OHARRAK.....	25
5.1. Lanaren ahulguneak.....	25
5.2. Etorkizunerako hobekuntzak.....	25

1. SARRERA

BookJournal aplikazioa liburuak irakurtzen dituzten erabiltzaileetan pentsatuz sortutako eguneroko digitala da, Android Studio ingurunean garatutakoa eta Java zein Kotlin programazio-lengoaiei bidez eraikia. Helburu nagusia erabiltzaile bakoitzak bere irakurketa ohiturak kontrolatzeko eta hobeto ulertzeko herramienta bat izatea da. Honetarako, aplikazioak erabiltzailearen datuak gordetzen ditu, aurrerago irakurritako liburuen inguruko informazioa jaso eta bistaratzeko.

Aplikazio honen bidez, erabiltzaileek beren irakurketa esperientzia hobeto antolatu dezakete, liburuei puntuazioa eman, desira-zerrendak sortu eta euren irakurketa ohituren gaineko ikuspegi bisual bat izan. Dokumentu honetan, aplikazioaren analisia, diseinua eta garapen-prozesua aurkeztuko ditugu, sortutako funtzionalitate guztiak xehatuz.

1.1. Erabilitako teknologia

KATEGORIA	TEKNOLOGIA	ERABILERA
Lengoiak	Java, Kotlin	Programazio-lengoiak
Garatze ingurunea	Android Studio	Aplikazioaren garapena
UI diseinua	XML layouts, ViewBinding	Pantailen diseinua eta elementuen lotura
Nabigazioa	Android Navigation Component	Fragment-en arteko nabigazioa
Autentifikazioa	Firebase Authentication	Erabiltzaileen kudeaketa (email/password eta Google Sign-In)
Datu Basea	Firebase Firestore	Datuen biltegitarte sinkronizatua
APIak	Open Library API	Liburuen bilaketa eta informazioa
	Open Library WorkDetails API	Liburuen sinopsiak eskuratzeko
Liburutegiak	Retrofit	API deietarako
	Glide	Irudiak kargatzeko
	Material Design Components	Material Design estiloak
	Credential Manager	Google sign-in modernoa
Gailuaren orientazioa	Portrait / Landscape	Bi orientazioetarako egokitzapena

2. APLIKAZIOAREN EZAUGARRIAK

Aplikazioa Android Studio erabiliz garatu da, MVVM (Model-View-ViewModel) arkitektura-eredua oinarritzat hartuta. Eredu honek interfazearen logika eta negozio-logika bereiztea ahalbidetzen du, kodearen mantentze-lanak eta eskalagarritasuna erraztuz. **Backend** zerbitzari gisa Firebase erabili da, bi funtzio nagusirekin:

- **Firestore Authentication:** erabiltzaileen autentifikazioa eta sesioen kudeaketa.
- **Cloud Firestore:** erabiltzaileen datuak (irakurritako liburuak, puntuazioak, desio-zerrendak) denbora errealean gordetzeko eta sinkronizatzeko.

Liburuen informazioa eta azalak lortzeko, Open Library API publikoa erabili da. API horretara konektatzeko, Retrofit liburutegia erabili da, Gson-ekin batera JSON erantzunak Java objektu bihurtzeko. Irudiak kargatzeko, berriz, Glide liburutegia erabili da, errendimendua eta cache kudeaketa optimizatzeko.

Datuen sinkronizazioa eta segurtasuna bermatzeko, aplikazioak Firebase-ren denbora errealeko eguneratzeak baliatzen ditu, eta erabiltzaile bakoitzaren datuak modu isolatuan gordetzen dira Firestore-n, segurtasun-arauak aplikatuz.

2.1 Autentifikazio Sistema

2.1.1 Hasierako Pantaila (FirstFragment)

Aplikazioa irekitzean erabiltzaileak ongietorri-pantaila bat aurkitzen du, non bi aukera nagusi eskaintzen diren: saioa hasi edo kontua sortu. Pantaila honek diseinu sinple eta erakargarria du, aplikazioaren logotipoarekin eta izenburuarekin.

- Ezaugarri garrantzitsua: **erabiltzailea dagoeneko autentifikatuta badago** (saioa irekita eta Firebase saioa aktibo), aplikazioak automatikoki HomeFragment-era nabigatzen du, hasierako pantaila erakutsi gabe. Honek erabiltzailearen esperientzia hobetzen du, ez baitu etengabe saioa hasi behar.

2.1.2 Erregistro Pantaila (SingUpFragment)

Erabiltzaile berriek kontua sor dezakete e-posta eta pasahitz bidez:

- **E-posta:** Baliozko e-posta helbidea.
- **Pasahitza:** Gutxienez 6 karaktere.
- **Pasahitza errepikatu:** Pasahitzak bat datozela egiaztatzeko.

Balioztatzeak:

- Pasahitzak gutxienez 6 karaktere izan behar ditu.
- Pasahitzak berdinak izan behar dira.
- Eremu guztiak bete behar dira.

Errore-mezuak:

- "Pasahitzak ez datoz bat".
- "Pasahitzak gutxienez 6 karaktere izan behar ditu".

2.1.3 Saio-hasiera Pantaila (LogInFragment)

Bi autentikazio-metodo eskaintzen dira:

1. E-posta eta pasahitzarekin:

Erabiltzaileak bere kredentzialak sartzen ditu eta "Login" botoian klik egiten du. Firebase Authentication-ek balioztatzen ditu kredentzialak.

2. Google kontuarekin:

Android-en **Credential Manager** erabiliz inplementatuta (deprecatutako GoogleSignInClient-en ordez). Abantailak:

- Segurtasun handiagoa.
- Erabiltzaileak gailuan dituen Google kontuak automatikoki detektatzen ditu.
- UI moderno eta integratua.

Google sign-in fluxua:

1. Erabiltzaileak "Login with Google" botoian klik egiten du.
2. Credential Manager-ek gailuko Google kontuen zerrenda erakusten du.
3. Erabiltzaileak kontu bat hautatzen du.
4. Google ID token-a lortzen da.
5. Firebase GoogleAuthProvider-rekin autentifikatzen da.
6. Firestore-n erabiltzailearen dokumentua sortzen da (beharrezkoa bada).

Kasu bereziak:

- Gailuan Google konturik ez badago, erabiltzaileari mezua erakusten zaio eta kontu bat gehitzera bideratzen da.
- Autentifikazioa ezeztatzen bada, errore-mezu argia erakusten da.

2.1.4 Saio-amaiera (Logout)

Aplikazioaren edozein atal nagusiatik egin daiteke saio-amaiera. **LogoutManager.kt** objektuak kudeatzen du prozesu hau:

1. Credential Manager-en egoera garbitu.
2. FirebaseAuth signOut() deitu.
3. Toast mezua erakutsi.
4. Nabigatu hasierako pantailara (FirstFragment)

Baieztapena eskatzen da saioa amaitu baino lehen, nahi gabeko amaierak saihesteko:

2.2 Nabigazioa eta Menuak

Aplikazioak nabigazio-sistema malgua eta intuitiboa du, gailuaren orientazioaren arabera egokitzen dena.

2.2.1 Nabigazioaren Egitura

NavGraph-ak 13 fragment ditu, ondorengo moduan antolatuta:

```
fristFragment (hasiera)
├── logInFragment
│   └── homeFragment
└── singUpFragment
    └── homeFragment
        ├── readingLogFragment
        │   ├── addBookLogFragment
        │   │   └── logBookInfoFragment
        │   └── logBookInfoFragment
        ├── readingSystemFragment
        ├── wishListFragment
        │   └── searchWishBookFragment
        ├── bookInfoFragment
        │   └── showBookInfoFragment
        └── favoriteBooksFragment
```

2.2.2 Menu Nagusiak

Bottom Navigation (bertikala - portrait):

5 atal nagusiak azkar aldatzeko:

- Irakurketa egunerokoa
- Estatistikak
- Desio-zerrenda
- Liburu informazioa
- Gogokoenak

Navigation Drawer (horizontala - landscape):

Orientazio horizontalean, Bottom Navigation desagertu eta FAB bat agertzen da alboko menua irekitzeko. Diseinu honi esker pantaila horizontalki dagoeneko eremuak aprobetxatzen dira. FAB-a sakatzean, Navigation Drawer irekitzen da, atal nagusietara sarbide azkarra emanez.

2.2.3 Nabigazioaren kudeaketa

MainActivity.java-k nabigazioa kontrolatzen du, fragment motaren arabera menuak erakutsi eta ezkutatzen:

```
navController.addOnDestinationChangeListener((controller, destination,
arguments) -> {
    int destId = destination.getId();

    // Fragment nagusiak (menuak ikusgai)
    if (destId == R.id.readingLogFragment ||
        destId == R.id.readingSystemFragment ||
        destId == R.id.wishListFragment ||
        destId == R.id.bookInfoFragment ||
        destId == R.id.favoriteBooksFragment) {

        updateMenuVisibilityByOrientation(); // Orientazioaren arabera
        drawerLayout.setDrawerLockMode(DrawerLayout.LOCK_MODE_UNLOCKED);

        // Autentikazio fragment-ak (menuak ezkutuan)
    } else if (destId == R.id.fristFragment ||
        destId == R.id.logInFragment ||
        destId == R.id.singUpFragment) {

        bottomNavigationView.setVisibility(View.GONE);
        fabMenu.setVisibility(View.GONE);
        drawerLayout.setDrawerLockMode(DrawerLayout.LOCK_MODE_LOCKED_CLOSED);
    }
});
```

Orientazio-aldaketa automatikoa:

```
public void onConfigurationChanged(@NonNull Configuration newConfig) {
    super.onConfigurationChanged(newConfig);

    if (fragment nagusietan bagaude) {
        updateMenuVisibilityByOrientation();
    }
}

private void updateMenuVisibilityByOrientation() {
    if (getResources().getConfiguration().orientation ==
    Configuration.ORIENTATION_LANDSCAPE) {
        bottomNavigationView.setVisibility(View.GONE);
        fabMenu.setVisibility(View.VISIBLE); // FAB agertu
    } else {
        bottomNavigationView.setVisibility(View.VISIBLE);
        fabMenu.setVisibility(View.GONE);    // FAB ezkutatu
    }
}
```

2.3 Irakurketa Egunerokoa (Reading Log)

Atal honetan erabiltzaileak irakurritako liburuak erregistratu ditzake, irakurketaren datu guztiekin.

2.3.1 Liburuen Bilaketa

Open Library API erabiliz, erabiltzaileak liburuak bilatu ditzake izenburuaren arabera:

- **Bilaketa automatikoa** hasten da 3 karaktere idaztean.
- **Debounce mekanismoa**: 1 segundoko atzerapena idazketaren amaieratik bilaketa exekutatu arte, API deiak optimizatzeko.
- **Emaitzak** RecyclerView batean erakusten dira.

2.3.2 Emaizten txartelak

Txartel bakoitzak erakusten du:

- Liburuaren azala (Glide-rekin kargatua).
- Izenburua.
- Egilea(k).
- "Gehitu" botoia.

Bilaketa-kodea:

```
private void setupSearchView() {
    binding.text.setOnQueryTextListener(new SearchView.OnQueryTextListener() {
        @Override
        public boolean onQueryTextChange(String s) {
            clearPendingSearches();

            if (s != null && s.trim().length() >= 3) {
                searchRunnable = () -> bookViewModel.search(s.trim());
                searchHandler.postDelayed(searchRunnable, 1000);
            }
            return false;
        }
    });
}
```

2.3.3 Irakurketa datuen sarrera

Liburua hautatu ondoren, erabiltzaileak irakurketaren datu guztiak betetzen ditu formulario batean:

Datuak:

- **Hasiera data eta Amaiera data:** DatePicker dialog bidez hautatzen dira.
- **Irakurketa mota:** Testu librekoa (fisikoa, digitala, audioa...).
- **Puntuazioa:** 5 izarreko sistema interaktiboa (Reading System atalarekin zerikuzia duena).
- **Oharrak:** Liburuaren inguruko oharrak idazteko eremua.

DatePicker implementazioa:

```
private void showDatePicker(boolean isStartDate) {
    Calendar calendar = Calendar.getInstance();

    DatePickerDialog datePickerDialog = new DatePickerDialog(
        requireContext(),
        (view, year, month, dayOfMonth) -> {
            String selectedDate = String.format("%02d/%02d/%d",
                dayOfMonth, month + 1, year);

            if (isStartDate) {
                binding.sDateValue.setText(selectedDate);
            } else {
                binding.eDateValue.setText(selectedDate);
            }
        }
    );
}
```

```

        },
        calendar.get(Calendar.YEAR),
        calendar.get(Calendar.MONTH),
        calendar.get(Calendar.DAY_OF_MONTH)
    );
    datePickerDialog.show();
}

```

Puntuazio-sistema:

```

private void setRating(int rating) {
    selectedRating = rating;

    for (int i = 0; i < starButtons.size(); i++) {
        ImageButton starButton = starButtons.get(i);
        if (i < rating) {
            starButton.setImageResource(R.drawable.ic_star_filled_24dp);
        } else {
            starButton.setImageResource(R.drawable.ic_star_border_24dp);
        }
    }
}
}

```

Balioztatzeak gorde aurretik:

- Puntuazioa hautatu behar da.
- Hasiera data bete behar da.
- Amaiera data bete behar da.
- Amaiera data ezin da hasiera data baino lehenagokoa izan.

2.3.4 Irakurketa erregistroen zerrenda

Erregistro bakoitzak erakusten du:

- Liburuaren azala.
- Izenburua eta egilea.
- Hasiera eta amaiera datak.
- Puntuazioa.
- "Editatu" botoia.

Firestore-ko egitura:

```
usuarios/{userId}/reading_logs/{documentId}
- title: String
- author: String
- isbn: String
- genre: String
- imageUrl: String
- bookId: String
- startDate: String
- endDate: String
- lectureType: String
- rating: int
- notes: String
- userId: String
- timestamp: long
```

Edizioa:

Erregistro bat editatzean, datuak automatikoki kargatzen dira formularioan eta aurretik "Save" zen botoia "Update" bihurtzen da. Aldaketak gordetzean, Firestore-n dokumentuaren eguneraketa gertatzen da.

2.4 Irakurketa Sistema (Reading System)

Atal honek erabiltzaileari aukera ematen dio **puntuazio (izar) bakoitzari balio pertsonalizatu bat esleitzeko**.

2.4.1 Atal honen helburua

Erabiltzaileak bere irakurketa-sistema propioa definitu dezake, non izar kopuru bakoitzak esanahi edo balio pertsonal bat duen. Adibidez:

- ★ = "Ez dut bat ere gomendatzen"
- ★★ = "ez zait gustiz gustatu"
- ★★★ = "Ez dago gaixki"
- ★★★★ = "Gustatu zait"
- ★★★★★ = "Oso gomendagarria berriz irakurriko nuke"

2.4.2 Pantailaren diseinua

Pantaila bi zati nagusitan banatzen da:

Ezkerraldea - Izarren balioak:

- ★ 1 izarren balioa (testu gisa)
- ★★ 2 izarren balioa
- ★★★ 3 izarren balioa
- ★★★★ 4 izarren balioa
- ★★★★★ 5 izarren balioa

Eskuinaldea - Datuak aldatzeko panela:

- **Spinner-a:** 1-5 izarak hautatzeko.
- **Testu-eremua:** Balio berria sartzeko (edozein testu).
- **"Edit Value" botoia:** Aldaketa gordetzeko.

2.4.3 Funtzionamendua

1. **Spinner-ean:** izar kopurua hautatzen da (1 ★, 2 ★, 3 ★, 4 ★ edo 5 ★)
2. **Testu-eremuan:** balio berria idazten da (adibidez, "Erraza", "Konplexua", "20 egun", etab.)
3. **"Edit Value" botoia** sakatzean:
 - Hautatutako izarren balioa eguneratzen da.
 - Firestore-n gordetzen da.
 - Pantailako testua automatikoki eguneratzen da.

```
private void updateStarValue(String star, String newValue) {  
    switch (star) {  
        case "1 ★":  
            readingData.setStar1Value(newValue);  
            txtStar1.setText(newValue);  
            break;  
        case "2 ★":  
            readingData.setStar2Value(newValue);  
            txtStar2.setText(newValue);  
            break;  
        // ... 3, 4, 5 izarrentzat ere bai  
    }  
    saveReadingSystemData();  
}
```

2.4.4 Firestore-ko egitura

Datuak Firebase Firestore-n gordetzen dira, erabiltzaile bakoitzarentzat:

```
usuarios/{userId}/reading_system/stats  
├── star1Value: String // "Koaderno txikia"  
├── star2Value: String // "Eleberri laburra"  
├── star3Value: String // "Saiakera"  
├── star4Value: String // "Nobela historikoa"  
├── star5Value: String // "Klasikoa"  
├── updatedAt: long    // 1705357200000  
└── userId: String     // "abc123..."
```

2.5 Desio-zerrenda (Wishlist)

Erabiltzaileek irakurri nahi dituzten liburuak gorde ditzakete zerrenda honetan, geroago erosteko edo irakurtzeko asmoz.

2.5.1 Liburuen bilaketa

Desio-zerrendarako liburuen bilaketa

Irakurketa egunerokoaren antzeko bilaketa-sistema, baina helburua desberdina da: hemen liburuak gordetzen dira desio-zerrendan, ez irakurritakoen erregistroan.

Bilaketa-ezaugarriak:

- **Bilaketa automatikoa:** 3 karaktere idaztean abiarazten da.
- **Debounce sistema:** segundu 1eko itxaron-denbora du idazketaren amaieran (API deiak optimizatzeko).
- **Emitzak *RecyclerView*-ean:** Txartel bakoitzean azala, izenburua eta egilea(k).

Bilaketa-emitzen txartela

Txartelaren elementuak:

- **Azala:** Glide-rekin kargatua (errorerako eta placeholder-erako irudi lehenetsiekin).
- **Izenburua:** Open Library-tik lortua.
- **Egilea(k):** Egile bakarra edo bat baino gehiago koma bidez banatuta.
- **“Gehitu” botoia:** Liburua desio-zerrendan gehitzeko.

```
// Egileak formateatzeko
if (volumeInfo.authors != null && !volumeInfo.authors.isEmpty()) {
    StringBuilder authorsText = new StringBuilder();
    for (String author : volumeInfo.authors) {
        if (authorsText.Length() > 0) authorsText.append(", ");
        authorsText.append(author);
    }
    holder.binding.authorID.setText(authorsText.toString());
}
```

Gordetze-prozesua

Erabiltzaileak "Gehitu" botoian klik egitean:

1. `WishlistItem` objektua sortzen da:

```
WishlistItem wishlistItem = new WishlistItem(
    volumeInfo.title,
    volumeInfo.authors,
    volumeInfo.coverImage,
    currentUser.getId()
);
```

2. Firestore-n gordetzen da:

```
db.collection("usuarios")
    .document(currentUser.getId())
    .collection("wishlist")
    .add(wishlistItem)
    .addOnSuccessListener(documentReference -> {
        wishlistItem.setId(documentReference.getId());
        wishlistViewModel.addWishlistItem(wishlistItem);
        Toast.makeText(requireContext(), R.string.book_saved,
            Toast.LENGTH_SHORT).show();
    });
```

3. **ViewModel eguneratzen da:** *WishlistViewModel*-eko lista eguneratzen da eta behatzaileei jakinarazten zaie.
4. **Toast mezua:** "Liburua gorde da" mezua erakusten da.

2.5.2 Desio-zerrendaren kudeaketa

Elementu bakoitzak erakusten du:

- **Azala:** Glide-rekin kargatua.
- **Izenburua:** Letra lodiz nabarmendua.
- **Egilea(k):** Egilearen izena(k).
- **"Ezabatu" botoia:** Elementua zerrendatik kentzeko.

WishlistAdapter-ak kudeatzen du *RecyclerView*:

```
public class WishlistAdapter extends
RecyclerView.Adapter<WishlistAdapter.WishlistViewHolder> {

    public interface OnDeleteClickListener {
        void onDeleteClick(WishlistItem item, int position);
    }

    @Override
    public void onBindViewHolder(@NonNull WishlistViewHolder holder, int
position) {
        WishlistItem item = wishlistItems.get(position);

        // Ezabatu botoiaren listenera
        holder.binding.btnWishDelete.setOnClickListener(v -> {
            if (deleteListener != null) {
                deleteListener.onDeleteClick(item, position);
            }
        });
    }
}
```

```

// Item osoaren klik listenera
holder.itemView.setOnClickListener(v -> {
    if (itemClickListener != null) {
        itemClickListener.onItemClick(item);
    }
});
}
}
}

```

Elementua ezabatzea:

Ezabatze-prozesua segurtasunez diseinatuta dago:

1. Erabiltzaileak "Ezabatu" botoian klik egiten du
2. Baieztapen-dialogoa agertzen da:
 - a. Izenburua: "Ezabatu"
 - b. Mezua: "Ziur zaude [liburuaren izena] ezabatu nahi duzula?"
 - c. "Bai" eta "Ez" botoiak
3. "Bai" sakatzean:

```

private void deleteItemFromFirestore(WishlistItem item, int position) {
    db.collection("usuarios")
        .document(currentUser.getId())
        .collection("wishlist")
        .document(item.getId())
        .delete()
        .addOnSuccessListener(aVoid -> {
            adapter.removeItem(position);
            wishlistViewModel.removeItem(position);
            Toast.makeText(requireContext(),
                R.string.delete_success,
                Toast.LENGTH_SHORT).show();
        });
}
}

```

4. Emaita:
 - a. Firestore-tik dokumentua ezabatzen da
 - b. RecyclerView-etik elementua kentzen da (animazioarekin)
 - c. Toast mezua: "Ezabatuta!"
 - d. ViewModel-eko lista eguneratzen da

Firestore-ko egitura:

```
usuarios/  
└─ {userId}/  
    └─ wishlist/  
        └─ {documentId}/  
            ├── title: String  
            ├── authors: List<String>  
            ├── coverImage: String  
            └─ userId: String
```

Datuen karga hasieratzean:

Fragment-a irekitzean, datuak automatikoki kargatzen dira:

```
private void loadWishlistItemsFromFirestore() {  
    db.collection("usuarios")  
        .document(currentUser.getId())  
        .collection("wishlist")  
        .get()  
        .addOnSuccessListener(queryDocumentSnapshots -> {  
            List<WishlistItem> items = new ArrayList<>();  
            for (DocumentSnapshot document : queryDocumentSnapshots) {  
                WishlistItem item = document.toObject(WishlistItem.class);  
                item.setId(document.getId());  
                items.add(item);  
            }  
            wishlistViewModel.setWishlistItems(items);  
        });  
}
```

2.6 Liburuen Informazioa (Book Information)

Atal honetan erabiltzaileek edozein libururi buruzko informazio zehatza bilatu eta ikusi dezakete, inongo erregistrorik egin gabe. Ezinbesteko tresna da liburu bat erosi edo irakurri aurretik informazioa eskuratzeko.

2.6.1 Bilaketa

Bilaketa-funtzionalitateak:

- Bilaketa automatikoa: 3 karaktere baino gehiago idaztean aktibatzen da
- Debounce: 1 segundoko atzerapena, API deiak optimizatzeko
- Emaizak RecyclerView-ean: Txartel bakoitzak liburu baten informazioa erakusten du

2.6.2 Emaizten txartelak

- **Azala:** Glide-rekin kargatua, cache estrategiarekin (`DiskCacheStrategy.ALL`).
- **Izenburua:** Letra lodiz nabarmendua.
- **Egilea(k):** Egile zerrenda.
- **"Select" botoia:** Xehetasunak ikusteko.

2.6.3 Sinopsien Karga

Liburu bakoitzaren sinopsia kargatzea da atal honen ezaugarri nagusietako bat. *Open Library-ren WorkDetails APIa* erabiltzen da horretarako:

```
public void fetchSynopsis(String key, Book.VolumeInfo volumeInfo) {  
    // Markatu kargatzen ari dela  
    Map<String, String> currentMap = synopsisMap.getValue();  
    currentMap.put(volumeInfo.title, "Kargatzen...");  
    synopsisMap.postValue(currentMap);  
  
    // API deia  
    Book.api.getWorkDetails(key).enqueue(new Callback<Book.WorkDetails>() {  
        @Override  
        public void onResponse(@NonNull Call<Book.WorkDetails> call,  
                                @NonNull Response<Book.WorkDetails> response) {  
            if (response.isSuccessful() && response.body() != null) {  
                String synopsis = response.body().getDescriptionText();  
                if (synopsis != null && !synopsis.isEmpty()) {  
                    map.put(volumeInfo.title, synopsis);  
                } else {  
                    map.put(volumeInfo.title, "Sinopsirik ez");  
                }  
            }  
        }  
    });  
}
```

Open Library-ren erantzun-egitura:

```
{  
    "title": "Gerra eta Bakea",  
    "description": "Nobelaren sinopsia hemen...",  
    "subjects": ["Nobela", "Errusia", "Gerra"],  
    "covers": [123456, 789012]  
}
```

Sinopsia lortzeko logika:

```
public String getDescriptionText() {
    if (description == null) return null;

    // Kasu 1: String zuzena bada
    if (description instanceof String) {
        return (String) description;
    }

    // Kasu 2: Objektua bada "value" eremuarekin
    if (description instanceof Map) {
        Map<String, Object> descMap = (Map<String, Object>) description;
        Object value = descMap.get("value");
        if (value instanceof String) {
            return (String) value;
        }
    }

    return null;
}
```

Sinopsien Mapa eta LiveData:

BookViewModel-ek **synopsisMap** bat mantentzen du, non liburuaren izenburua sinopsiarekin lotzen den:

```
private MutableLiveData<Map<String, String>> synopsisMap = new
MutableLiveData<>(new HashMap<>());

public MutableLiveData<Map<String, String>> getSynopsisMap() {
    return synopsisMap;
}
```

2.6.4 Xehetasunen Pantaila

Erabiltzaileak "Select" botoian klik egitean, **ShowBookInfoFragment** irekitzen da, liburuaren informazio guztiarekin:

```
Bundle bundle = new Bundle();
bundle.putString("book_title", volumeInfo.title);
bundle.putStringArray("book_authors", volumeInfo.authors.toArray(new
String[0]));
bundle.putString("book_cover", volumeInfo.coverImage);
bundle.putString("book_published", volumeInfo.publishedDate);
bundle.putString("book_isbn", volumeInfo.isbns.get(0));
bundle.putString("book_synopsis", synopsis);
```

Android-en fragment-en arteko komunikaziorako hainbat metodo daude, baina kasu honetarako Bundle erabiltzeak abantaila garrantzitsuak ditu:

1. Fragment-en izaera independentea: Bundle bat erabiltzean, fragment-ek ez dute elkarren erreferentziak gorde behar, independenteak dira eta birsor daitezke.
2. Birsortze-prozesuetan iraunkortasuna: Android-en gerta daitezkeen egoera-alaketetan (pantailaren biratzea, hizkuntza-alaketa, memoriaren kudeaketa) datuak ez dira galtzen, savedInstanceState-en bidez berreskuratzen da.
3. Android-en diseinu-filosofia: Google-k gomendatzen duen modua da fragment-en arteko komunikaziorako:

➤ "You should pass only the minimum required data between fragments using Bundle arguments." - Android Developers

Zergatik?

- Mantentze erraza: Datu-fluxua argia eta dokumentatua dago.
- Testeagarritasuna: Fragment-ak isolatuta proba daitezke.
- Modularitatea: Fragment-ek ez dute elkarren implementazio-ezagutzarik behar.

Goiko zatia - **CardView** nagusia:

- Azala: Tamaina handian, ezkerrean.
- Izenburua: Letra lodiz.
- Egilea(k): Egilearen izenak.
- ISBN: Nazioarteko zenbaki normalizatua.
- Argitaratze-urtea: Lehen argitaratzearen data.

Beheko zatia - **Sinopsia**:

- Sinopsia: Testu luzea, ScrollView batean irakurterraza izateko.
- Liburuaren deskribapen osoa, Open Library-tik eskuratua.

```

<ScrollView
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/tvSinopsis"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:lineSpacingExtra="2dp" />
</ScrollView>

```

Kudeaketa eta Memoria: *onDestroyView* metodoa erabiltzen da “memory leak”-ak sahiesteko:

```

@Override
public void onDestroyView() {
    super.onDestroyView();
    binding = null; // Binding-a askatu
}

```

2.7 Liburu Gogokoenak (Favorite Books)

Atal honek automatikoki erakusten ditu erabiltzaileak 4 eta 5 izarrekin baloratu dituen liburuak. Bereztasun nagusia: **ez da eskuz kudeatzen**, irakurketa-erregistroetatik automatikoki sortzen da.

2.7.1 Funtzionamendu Automatikoa

Nola funtzionatzen duen:

```

// 1. Erabiltzaileak irakurketa-erregistro bat gordetzen du 4-5 izarrekin
ReadingLog newLog = new ReadingLog();
newLog.setTitle("Gerra eta Bakea");
newLog.setRating(5); // 5 izar
readingViewModel.saveReadingLog(newLog);

// 2. FavoriteBooksFragment-ek automatikoki detektatzen du
public LiveData<List<FavoriteBooks>> getFavoriteBooks(String userId) {
    db.collection("usuarios")
        .document(userId)
        .collection("reading_logs")
        .orderBy("rating", Query.Direction.DESENDING)

```

```

        .get()
        .addOnCompleteListener(task -> {
            List<FavoriteBooks> favBooks = new ArrayList<>();
            for (QueryDocumentSnapshot document : task.getResult()) {
                FavoriteBooks favBook =
document.toObject(FavoriteBooks.class);
                if (favBook.getRating() > 4) { // 4 eta 5 izarrekoak soilik
                    favBook.setDocumentId(document.getId());
                    favBooks.add(favBook);
                }
            }
            favsLiveData.setValue(favBooks); // 3. UI eguneratu
        });
    }
}

```

2.7.2 ReadingLog-etik FavoriteBooks-era

ReadingLog objektuak datu hauek ditu:

```

public class ReadingLog {
    private String title;        // "Gerra eta Bakea"
    private String author;       // "Lev Tolstoi"
    private String isbn;         // "978-84-1234-56-7"
    private String genre;        // "Nobela"
    private String imageUrl;     // "https://..."
    private String bookId;       // "/works/OL15395W"
    private int rating;          // 5
    // ... beste eremuak
}

```

FavoriteBooks objektuak soilik beharrezkoak diren eremuak gordetzen ditu:

```

public class FavoriteBooks {
    private String title;        // "Gerra eta Bakea"
    private String author;       // "Lev Tolstoi"
    private String imageUrl;     // "https://..."
    private String bookId;       // "/works/OL15395W"
    private int rating;          // 5
    private String userId;       // "abc123..."
    private String documentId;   // Firestore ID-a
}

```

2.7.3 Pantailaren diseinua

FavoriteBooksFragment-aren layout-a:

```
<FrameLayout>
    <LinearLayout>
        <LinearLayout android:height="45dp" />
        <LinearLayout android:padding="16dp">
            <LinearLayout>
                <Button android:id="@+id/btnLogOut" />
                <TextView android:text="@string/favorites_book_title" />
            </LinearLayout>
            <androidx.recyclerview.widget.RecyclerView
                android:id="@+id/recyclerviewFBcontents"
                app:layoutManager=
"androidx.recyclerview.widget.GridLayoutManager"
                app:spanCount="1" /> <!-- portrait: 1 zutabe -->
            </LinearLayout>
        </LinearLayout>
    </FrameLayout>
```

ItemFavoriteBookBinding txartelaren egitura:

```
<androidx.cardview.widget.CardView>
    <LinearLayout android:orientation="horizontal">
        <!-- Azala -->
        <LinearLayout android:layout_width="100dp">
            <ImageView android:id="@+id/image2" />
        </LinearLayout>

        <!-- Informazioa -->
        <LinearLayout android:orientation="vertical">
            <TextView android:id="@+id/titleID" /> <!-- Izenburua -->
            <TextView android:id="@+id/authorID" /> <!-- Egilea -->

            <!-- Puntuazioa -->
            <LinearLayout>
                <TextView text="★ " />
                <TextView android:id="@+id/ratingID" /> <!-- 4/5 edo 5/5 -->
            </LinearLayout>
        </LinearLayout>
    </LinearLayout>
</androidx.cardview.widget.CardView>
```

3. APLIKAZIOKO KODIGOAREN EGITURA

```
eus.arreseainhize.bookjournal/  
├─ MainActivity.java # Aktivitate nagusia (menuak, nabigazioa)  
├─ fragments/  
│   ├─ auth/  
│   │   ├─ FristFragment.java # Ongietorri pantaila  
│   │   ├─ LogInFragment.java # Saio-hasiera (Google + email)  
│   │   └─ SingUpFragment.java # Erregistroa  
│   └─ home/  
│       └─ HomeFragment.java # Hasiera menua  
├─ reading/  
│   ├─ ReadingLogFragment.java # Irakurketa erregistroak  
│   ├─ AddBookLogFragment.java # Liburua bilatu eta gehitu  
│   ├─ LogBookInfoFragment.java # Irakurketa datuak bete  
│   └─ ReadingSystemFragment.java # Sistema  
├─ wishlist/  
│   ├─ WishListFragment.java # Desio-zerrenda  
│   └─ SearchWishBookFragment.java # Liburuak bilatu  
├─ books/  
│   ├─ BookInfoFragment.java # Liburu informazioa bilatu  
│   ├─ ShowBookInfoFragment.java # Xehetasunak ikusi  
│   └─ FavoriteBooksFragment.java # Gogokoenak  
├─ adapters/  
├─ WishlistAdapter.java # Wishlist RecyclerView adapterra  
├─ StarRatingAdapter.java # Izarren spinnera  
├─ viewmodels/  
│   ├─ BookViewModel.java # Open Library API  
│   ├─ ReadingViewModel.java # Irakurketa logika  
│   ├─ UserViewModel.java # Erabiltzailearen egoera  
│   └─ WishlistViewModel.java # Wishlist logika  
├─ models/  
│   ├─ Book.java # Open Library modeloak  
│   ├─ ReadingLog.java # Irakurketa erregistroa  
│   ├─ ReadingSystem.java # Estatistika datuak  
│   ├─ WishlistItem.java # Desio-zerrendako elementua  
│   ├─ FavoriteBooks.java # Gogoko liburua  
│   └─ User.java # Erabiltzailea  
├─ utils/  
│   ├─ FirebaseAuthHelper.java # Firebase Auth utilitateak  
│   ├─ FirestoreHelper.java # Firestore funtzioak  
│   ├─ FirestoreOfflineHelper.java # Offline sinkronizazioa  
│   └─ GoogleSignInHelper.kt # Google sign-in Kotlin-ean
```

```

|   ├── LogoutManager.kt # Saio-amaiera kudeatzailea
|   ├── StringResourceHelper.java # String baliabideak
|   └── SharedPreferencesHelper.java # (Erabili gabe)
└── res/
    ├── layout/ # XML layout fitxategiak
    |   ├── activity_main.xml
    |   ├── fragment_.xml (13 fitxategi)
    |   ├── item_.xml (5 fitxategi)
    |   └── nav_header.xml
    ├── menu/
    |   ├── bottom_menu.xml
    |   └── drawer_menu.xml
    ├── navigation/
    |   └── nav_graph.xml
    ├── drawable/ # Irudi eta baliabideak
    └── values/ # String, kolore eta estiloak

```

4. OINARRIZKO FUNTZIONAMENDUA

4.1. Autentifikazio-fluxua

1. **FristFragment**: Erabiltzailea autentifikatuta badago, zuzenean HomeFragment-era nabigatzen da.
2. **LoginFragment**:
 - a. Email/password bidezko saio-hasiera
 - b. Google sign-in Credential Manager bidez
3. **SingUpFragment**: Erabiltzaile berria sortu
4. **Firestore**: Autentikazioaren ondoren, erabiltzailearen dokumentua sortzen da "usuarios" bilduman

4.3. Nabigazioaren kudeaketa

- **MainActivity**: NavController-aren bidez fragment-en arteko nabigazioa kontrolatzen du
- **Destination Change Listener**: Fragment motaren arabera menuak ezkutatu/erakutsi:
 - Fragment nagusiak (readingLog, readingSystem, etc.) → BottomNav eta FAB erakutsi.
 - Autentikazio fragment-ak → Menu guztiak ezkutatu.
- **Orientazio aldaketa**: onConfigurationChanged() metodoak menuak eguneratzen ditu

4.3. Open Library API integrazioa

1. **BookViewModel.search()**: Liburuak bilatzen ditu search.json endpoint-ean
2. **BookViewModel.fetchSynopsis()**: WorkDetails eskatzen ditu sinopsia lortzeko
3. **Retrofit**: API deiak egiteko erabiltzen da

4.4. Firestore datu-egitura

```
usuarios/  
└─ {userId}/  
  ├─ reading_logs/ # Irakurketa erregistroak  
  │   └─ {documentId}  
  ├─ wishlist/ # Desio-zerrenda  
  │   └─ {documentId}  
  └─ reading_system/ # Estatistika datuak  
      └─ stats
```

5. OHARRAK

5.1. Lanaren ahulguneak

1. **Offline funtzionalitatea**: Firestore-k offline sinkronizazioa onartzen duen arren, erabiltzaile-dokumentuak sortzeko logikan hobekuntzak behar ditu.
2. **Sinopsien karga**: Open Library API-tik sinopsiak kargatzeko prozesua ez da optimoa; batzuetan huts egiten du.
3. **Datuen balidazioa**: Sarrera-datuen balidazioa (data formatua, esaterako) hobetu daiteke.
4. **Google Sign-In erroreak**: Credential Manager-ekin erroreak gertatzen direnean, erabiltzaileari ez zaio mezu argirik erakusten.
5. **FavoriteBooks**: 4-5 izarreko liburuak automatikoki gehitzen dira, baina ezin dira eskuz kudeatu.

5.2. Etorkizunerako hobekuntzak

1. **UI/UX hobekuntzak**:
 - a. Animazio gehigarriak nabigazioan.
 - b. Material Design 3 gida-lerroak bete.
 - c. Iluna modua (Dark mode) gaitu.
2. **Funtzionalitate berriak**:
 - a. Irakurketa helburuak ezarri.
 - b. Urteko irakurketa estatistika bisualak (grafikoak).
 - c. Liburuen gomendioak irakurritakoen arabera.
 - d. QR kode bidezko ISBN eskaneatzea.
 - e. Liburu klubak kudeaketa.

3. Errendimendu hobekuntzak:

- a. Paging 3 liburutegia erabili RecyclerView-entzat.
- b. Cache sistema hobetu irudietarako.
- c. Datu-base lokal bat (Room) offline funtzionalitate hobeturako.

4. Segurtasuna:

- a. Bi faktoreko autentikazioa (2FA).
- b. Datuen enkriptazioa lokalean.
- c. Firestore segurtasun arauak optimizatu.

5. Kodearen kalitatea:

- a. Test unitario gehiago.
- b. UI test-ak.